INFORME MAESTRO DE ARQUITECTURA INICIAL

SUITE EMPRESARIAL TELCOBRAS

Documento: Arquitectura conceptual, funcional y técnica para iniciar el desarrollo e investigación con Claude Code en Visual Studio Code

Versión: 0.1 — Arquitectura general inicial

Fecha: 3 de noviembre de 2026

Producto: Suite Empresarial Telcobras

Responsable de arquitectura y dirección del proyecto: Arquitecto del producto

Enfoque: Plataforma empresarial modular, multiempresa, con inteligencia comercial, automatización documental, CRM e inteligencia artificial.

1. VISIÓN GENERAL DEL PROYECTO

1.1. ¿Qué es Suite Empresarial Telcobras?

Suite Empresarial Telcobras será una plataforma de software empresarial que centralizará herramientas de gestión comercial, captación de clientes potenciales, administración documental, automatización de procesos e inteligencia artificial.

No se pretende construir únicamente un CRM, una aplicación de facturación o un conjunto de automatizaciones independientes.

La visión es desarrollar un ecosistema empresarial integrado, compuesto por módulos especializados que puedan funcionar individualmente, pero que compartan información, servicios, infraestructura y reglas de negocio.

La suite tendrá una aplicación web central desde la cual cada empresa podrá administrar sus operaciones y utilizar las funcionalidades habilitadas para su organización.

La arquitectura deberá permitir que Telcobras incorpore nuevos módulos, integraciones y capacidades sin tener que reconstruir el sistema completo.

Objetivos principales

1. Centralizar las herramientas empresariales en una única plataforma.
2. Integrar LeadGen para investigación y captación de prospectos.
3. Incorporar funcionalidades RADIAN relacionadas con documentos electrónicos de la DIAN.
4. Incorporar un CRM para administrar clientes, empresas, oportunidades y actividades.
5. Integrar sistemas externos como Odoo, WhatsApp, servicios de correo y almacenamiento documental.
6. Incorporar posteriormente agentes de inteligencia artificial especializados por industria.
7. Permitir que diferentes empresas utilicen la plataforma de forma independiente y segura.
8. Mantener una arquitectura modular que permita desarrollar, actualizar y sustituir componentes.
9. Reducir progresivamente la dependencia de plataformas externas de automatización.
10. Convertir las herramientas que actualmente existen en activos tecnológicos integrados de Telcobras.

1.2. Principio fundamental de arquitectura

La suite debe funcionar como un sistema central que coordina módulos especializados.

No debemos desarrollar una aplicación monolítica en la que toda la lógica comercial, documental, financiera y de inteligencia artificial esté mezclada.

Tampoco debemos construir aplicaciones independientes que no puedan compartir información correctamente.

La arquitectura propuesta es una plataforma modular con servicios desacoplados y contratos de comunicación definidos. Esto significa:

- Cada módulo tendrá responsabilidades delimitadas.
- Los módulos no deberán acceder directamente a las bases de datos internas de otros módulos.
- La comunicación se realizará mediante APIs y eventos controlados.
- Los servicios compartirán una identidad empresarial y un sistema común de permisos.
- La aplicación web será la interfaz central de administración.
- Los procesos especializados podrán ejecutarse en servicios independientes.
- La infraestructura permitirá incorporar nuevos módulos sin afectar innecesariamente los existentes.

2. ESTADO ACTUAL DE LOS ACTIVOS TECNOLÓGICOS

Antes de definir la arquitectura futura, es importante identificar qué existe actualmente. Telcobras ya cuenta con herramientas funcionales o parcialmente integradas. No partimos de cero.

2.1. LeadGen

Estado conocido: herramienta funcional en entorno local. Integración de funcionalidades adicionales aproximadamente al 80 %.

Características conocidas:

- Desarrollada principalmente en Python, aunque el stack completo está pendiente de inspección.
- Tiene backend y API propios.
- Realiza investigación web para encontrar prospectos.
- Consulta diferentes fuentes y servicios.
- Cuenta con varias integraciones.
- No está desplegada actualmente en la nube.
- Se está unificando con funcionalidades de una herramienta denominada Harvey, orientadas también a la captación y contacto comercial por correo electrónico.

Objetivo dentro de la suite

LeadGen debe convertirse en un módulo integrado de inteligencia comercial. No debe permanecer como una aplicación independiente que el usuario tenga que abrir por fuera de la suite.

Su responsabilidad será encontrar, investigar, enriquecer y clasificar prospectos, y permitir que estos ingresen al ecosistema comercial.

> **Importante:** no se debe reescribir LeadGen antes de inspeccionar su repositorio, arquitectura, API, dependencias y funcionalidades actuales. La primera integración debe aprovechar lo que ya funciona.

2.2. RADIAN / DIAN

Estado conocido: herramienta funcional para descargar facturas y documentos relacionados con DIAN/RADIAN.

También existe una automatización construida con Make.com que:

1. Recibe documentos o información obtenida.
2. Extrae información de las facturas.
3. Procesa los datos.
4. Los envía a Odoo.
5. Crea borradores que posteriormente requieren aprobación y finalización manual.

La integración general está aproximadamente al 80 %. No se ha confirmado todavía el stack interno de RADIAN ni todos los detalles de sus conexiones con DIAN.

Objetivo dentro de la suite

RADIAN debe convertirse en un módulo de inteligencia documental y automatización administrativa. La suite deberá permitir consultar documentos, procesarlos, visualizar sus datos y conectarlos con los procesos administrativos de la empresa.

La integración con Odoo debe conservar inicialmente el control humano sobre la aprobación de documentos y operaciones. No se debe asumir que una extracción realizada por IA es suficiente para autorizar automáticamente una operación contable.

2.3. Agentes de inteligencia artificial

Estado conocido: existen agentes funcionales o parcialmente desarrollados mediante escenarios independientes en Make.com. Se orientan principalmente a:

- Atención al cliente.
- Ventas.
- Consulta de información.
- Automatización de procesos empresariales.
- Integración con CRM.
- Integración con ERP.
- Integración con Shopify.
- Consulta de documentos en Google Drive y OneDrive.
- Conexión con bases de datos.
- Integración con Odoo.
- Comunicación mediante WhatsApp y otros canales.

El desarrollo actual está aproximadamente al 80 %.

Objetivo dentro de la suite

Los agentes serán una funcionalidad integrada de la plataforma. Los usuarios finales no construirán agentes desde cero.

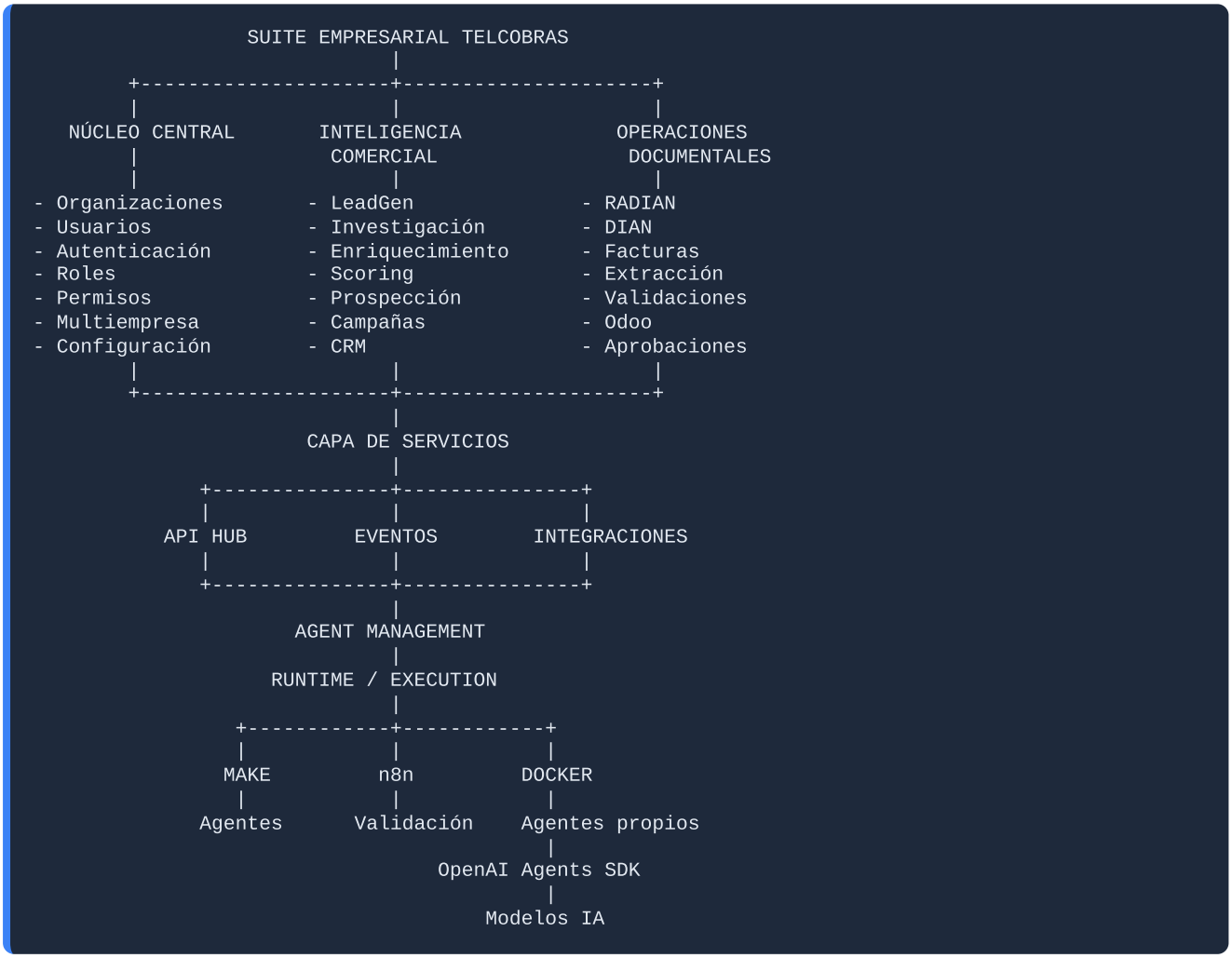
Telcobras desarrollará agentes especializados y los pondrá a disposición de las empresas según su industria y necesidades. Los usuarios podrán seleccionar agentes disponibles y personalizar determinados parámetros, por ejemplo:

- Nombre o identificación.
- Tono de comunicación.
- Instrucciones configurables dentro de límites establecidos.
- Documentos de conocimiento.
- Horarios de atención.
- Canales habilitados.
- Integraciones autorizadas.
- Herramientas disponibles.
- Parámetros comerciales.

La lógica interna y las capacidades fundamentales de cada agente estarán controladas por Telcobras.

3. MAPA CONCEPTUAL GENERAL DE LA SUITE

La plataforma se dividirá conceptualmente en grandes dominios funcionales. Estos dominios agrupan los módulos que deberán existir dentro del ecosistema.



Este esquema representa los dominios principales, no todos los módulos internos. A continuación se explica cada uno.

4. MÓDULO 1: NÚCLEO CENTRAL DE LA PLATAFORMA

Este será el componente fundamental de toda la suite. Su responsabilidad será administrar la identidad de las empresas, los usuarios, los permisos, las configuraciones y las condiciones de utilización de la plataforma.

Todos los módulos dependerán de este núcleo para identificar quién está utilizando el sistema y qué operaciones tiene autorizadas.

4.1. Gestión de organizaciones (Organization Management)

La plataforma deberá soportar múltiples empresas. Cada empresa tendrá su propio espacio lógico de trabajo.

Concepto:

```
SUITE TELCOBRAS
      |
      +---- Empresa A
      |        |
      |        +---- Usuarios
      |        +---- CRM
      |        +---- LeadGen
      |        +---- Documentos
      |        +---- Agentes
      |
      +---- Empresa B
      |        |
      |        +---- Usuarios
      |        +---- CRM
      |        +---- LeadGen
      |        +---- Documentos
      |        +---- Agentes
      |
      +---- Empresa C
               |
               +---- Usuarios
               +---- CRM
               +---- Documentos
               +---- Agentes
```

Cada organización deberá tener:

- Identificador único.
- Nombre comercial.
- Razón social, cuando corresponda.
- NIT u otro identificador empresarial.
- Información de contacto.
- Configuración regional.

- Zona horaria.
- Estado de la organización.
- Módulos habilitados.
- Configuraciones particulares.

Multi-tenancy

La arquitectura debe ser multi-tenant. Esto significa que varias empresas utilizarán la misma plataforma, pero sus datos estarán aislados. La separación debe aplicarse a:

- Clientes.
- Prospectos.
- Usuarios.
- Documentos.
- Conversaciones.
- Configuraciones.
- Integraciones.
- Credenciales.
- Agentes.
- Historiales.
- Reportes.

No será suficiente con ocultar información en el frontend. La separación debe implementarse también en backend y base de datos.

PostgreSQL ofrece Row-Level Security (RLS), que permite definir políticas para controlar qué registros puede consultar o modificar cada usuario. Puede utilizarse como una segunda barrera de aislamiento entre organizaciones. PostgreSQL

4.2. Gestión de usuarios

Cada empresa podrá tener múltiples usuarios. Información principal:

- ID.
- Nombre.
- Correo electrónico.
- Estado.
- Organización asociada.
- Roles.
- Fecha de creación.
- Último acceso.

El usuario deberá estar asociado a una organización, salvo cuentas administrativas internas con permisos especiales.

4.3. Autenticación

Responsabilidades:

- Inicio de sesión.
- Cierre de sesión.
- Recuperación de contraseña.
- Gestión de sesiones.
- Renovación de tokens.
- Verificación de identidad.
- Control de acceso.

Se propone utilizar autenticación basada en tokens de acceso y renovación, con almacenamiento seguro de credenciales y sesiones. La implementación concreta deberá definirse después de evaluar las necesidades de seguridad y los mecanismos de autenticación disponibles.

4.4. Roles y permisos (RBAC)

Se propone implementar Role-Based Access Control.

Ejemplo:

Rol	Responsabilidades
Superadministrador Telcobras	Administración global de la plataforma.
Administrador de empresa	Administración de su organización.
Gerente	Acceso a información y reportes autorizados.
Comercial	Gestión de leads, clientes y oportunidades.
Administrativo	Gestión documental y procesos administrativos.
Contador	Consulta y revisión de información contable.
Operador	Ejecución de operaciones autorizadas.
Usuario básico	Acceso limitado a funcionalidades.

Los roles deberán ser configurables. No debemos depender exclusivamente de nombres de roles predefinidos.

También se requiere autorización por recurso y acción. Por ejemplo:

- Consultar un cliente.
- Crear un cliente.
- Modificar un cliente.
- Eliminar un cliente.
- Consultar documentos.
- Aprobar documentos.
- Ejecutar un agente.
- Administrar integraciones.

4.5. Configuración de organizaciones

Cada empresa tendrá parámetros propios:

- Datos generales.
- Preferencias.
- Zona horaria.
- Moneda.
- Configuración de notificaciones.
- Integraciones.
- Módulos habilitados.
- Políticas internas.
- Preferencias de agentes.

4.6. Auditoría

Todas las operaciones importantes deberán poder rastrearse. Ejemplos:

- Usuario que creó un cliente.
- Usuario que modificó un documento.
- Usuario que aprobó un borrador.
- Agente que ejecutó una operación.
- Integración que envió información.
- Fecha y hora de cada operación.
- Resultado obtenido.

La auditoría no debe confundirse con los logs técnicos. La auditoría registra acciones empresariales; los logs registran el comportamiento técnico de los servicios.

5. MÓDULO 2: CRM EMPRESARIAL

El CRM será el centro de gestión comercial. Su responsabilidad será organizar las relaciones entre empresas, clientes, prospectos, oportunidades y actividades comerciales. Debe integrarse directamente con LeadGen.

5.1. Gestión de contactos

Permite almacenar personas relacionadas con clientes o empresas. Datos posibles:

- Nombre.
- Apellido.
- Correo electrónico.
- Teléfono.
- Cargo.
- Empresa asociada.
- Origen.
- Estado.
- Etiquetas.
- Observaciones.

5.2. Gestión de empresas

Permite registrar organizaciones que sean clientes, prospectos o aliados comerciales. Datos:

- Nombre.
- Razón social.
- NIT.
- Sector.
- Tamaño.
- Dirección.
- Sitio web.
- Contactos relacionados.
- Estado comercial.

5.3. Gestión de leads

Un lead representa un posible cliente. El CRM deberá recibir prospectos provenientes de:

- LeadGen.
- Formularios web.
- Importaciones.
- Campañas.
- Integraciones externas.
- Registro manual.

Cada lead tendrá un origen identificable. Se debe evitar que la misma persona o empresa aparezca repetida innecesariamente. Por eso se requiere un mecanismo de detección de duplicados.

5.4. Gestión de oportunidades

Las oportunidades representan procesos comerciales que pueden terminar en una venta. Ejemplo de pipeline:

```
NUEVO LEAD
    |
    v
CONTACTADO
    |
    v
CALIFICADO
    |
    v
REUNIÓN
    |
    v
PROPUESTA
    |
    v
NEGOCIACIÓN
    |
    +-----------> GANADO
    |
    +-----------> PERDIDO
```

Las etapas deberán poder configurarse.

5.5. Actividades comerciales

El CRM permitirá registrar:

- Llamadas.
- Correos.
- Reuniones.
- Seguimientos.
- Tareas.
- Notas.
- Compromisos comerciales.

5.6. Historial de interacción

Cada cliente deberá tener un historial de sus interacciones. Cuando exista una conversación mediante un agente de IA, la suite podrá registrar la interacción en el CRM. Esto permitirá que los usuarios autorizados conozcan el contexto comercial.

5.7. Reportes comerciales

Indicadores posibles:

- Leads obtenidos.
- Leads contactados.
- Leads calificados.
- Oportunidades abiertas.
- Oportunidades ganadas.
- Oportunidades perdidas.
- Tasa de conversión.
- Tiempo promedio de cierre.
- Origen de prospectos.

6. MÓDULO 3: LEADGEN — INTELIGENCIA COMERCIAL

LeadGen será el motor de investigación y captación de prospectos. Su función no será administrar todo el proceso comercial. Su función será encontrar información y entregar prospectos enriquecidos al CRM.

Esta separación es importante. LeadGen descubre oportunidades; CRM administra las relaciones comerciales.

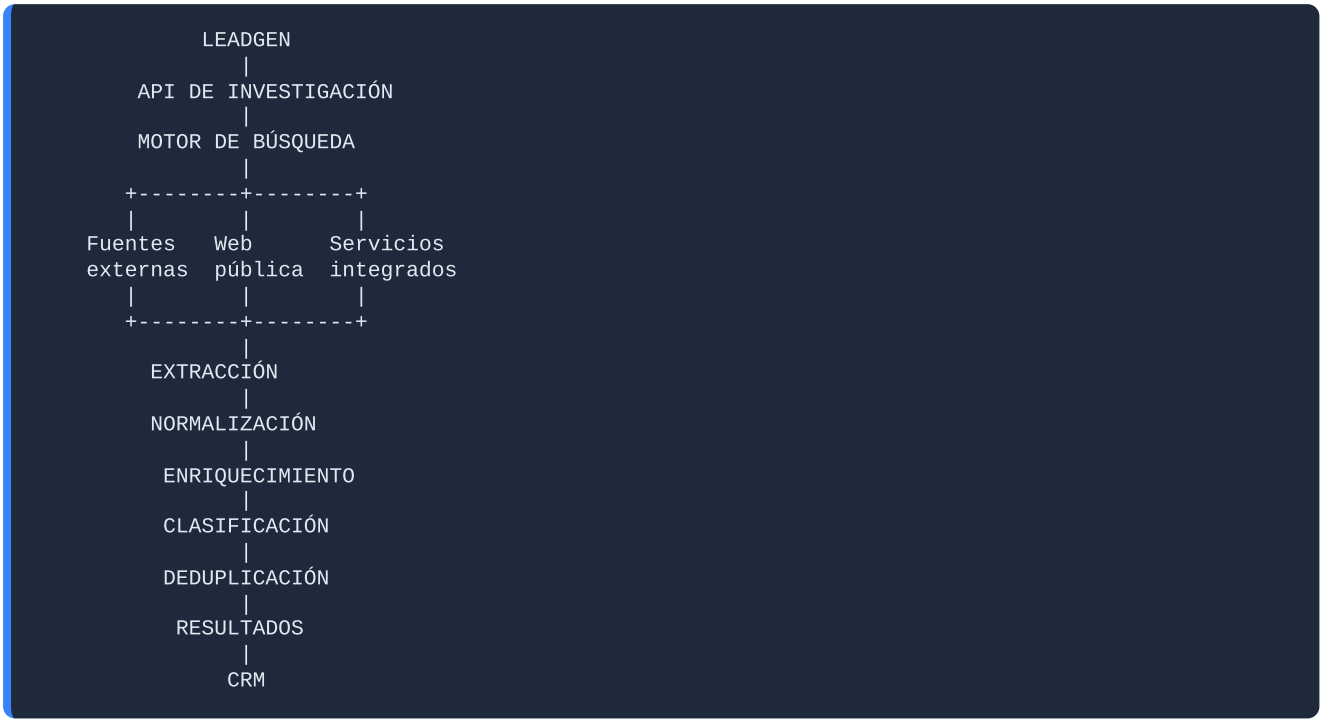
6.1. Responsabilidades

LeadGen deberá encargarse de:

1. Recibir criterios de búsqueda.
2. Identificar fuentes de información.
3. Consultar fuentes disponibles.
4. Extraer información empresarial.
5. Normalizar los datos.
6. Detectar duplicados.
7. Enriquecer registros.
8. Clasificar prospectos.

9. Calcular puntuaciones cuando exista un modelo definido.
10. Entregar resultados al CRM.
11. Permitir campañas de contacto autorizadas.

6.2. Arquitectura interna conceptual



La arquitectura real dependerá de lo que se encuentre al inspeccionar el repositorio actual. No se deben inventar componentes que ya existan ni reemplazar funcionalidades funcionales sin justificación.

6.3. Integración con CRM

LeadGen debe entregar los resultados mediante una API o contrato de integración. Ejemplo conceptual:

```
{
  "source": "leadgen",
  "company": {
    "name": "Empresa Ejemplo",
    "website": "https://example.com",
    "industry": "Servicios"
  },
  "contact": {
    "name": "Contacto Ejemplo",
    "email": "contacto@example.com"
  },
  "classification": {
    "score": 85,
    "category": "high_potential"
  }
}
```

Este ejemplo no representa el esquema definitivo. El CRM será responsable de crear o actualizar los registros correspondientes.

6.4. Campañas de contacto

La funcionalidad de contacto comercial por correo electrónico debe mantenerse separada de la investigación. Deberá contemplar:

- Plantillas.
- Personalización.
- Registro de envíos.
- Control de frecuencia.
- Estado de entrega.
- Rebotes.
- Respuestas.
- Exclusiones.
- Consentimiento y otras bases legales aplicables.

No se debe diseñar un sistema de envío masivo sin límites. La captación y el contacto de personas requieren revisar las obligaciones legales aplicables al tratamiento de datos personales y las condiciones de cada canal.

En Colombia, la Ley 1581 de 2012 se aplica independientemente de la tecnología utilizada para recolectar y tratar datos personales. La SIC también ha señalado expresamente su aplicación al uso de inteligencia artificial y tecnologías automatizadas. Sede Electrónica

7. MÓDULO 4: RADIAN / DOCUMENT INTELLIGENCE

Este módulo estará orientado al procesamiento y gestión documental empresarial. Su primera implementación estará relacionada con las funcionalidades existentes de RADIAN y DIAN.

7.1. Responsabilidades

- Consultar y descargar documentos mediante los mecanismos autorizados.
- Organizar documentos.
- Extraer información.
- Normalizar datos.
- Validar campos.
- Identificar inconsistencias.
- Asociar documentos con empresas y terceros.
- Enviar información a sistemas administrativos.

- Mantener trazabilidad.
- Permitir revisión y aprobación humana.

7.2. Arquitectura conceptual



7.3. Submódulos internos

A. Conector DIAN/RADIAN
Encapsula la comunicación con los servicios externos utilizados para obtener documentos. No se debe asumir cómo está implementado hasta revisar el código existente.

B. Document Manager
Gestiona los archivos:

- Identificación.
- Tipo.
- Empresa propietaria.
- Origen.
- Fecha.
- Estado.
- Ubicación de almacenamiento.
- Hash o identificador de integridad.

C. Extraction Engine
Extrae información de los documentos. Podrá utilizar diferentes técnicas dependiendo del formato:

- Lectura estructurada.
- Procesamiento XML.
- Extracción de texto.
- OCR cuando sea necesario.
- Modelos de IA para casos específicos.

La IA no debe reemplazar las validaciones determinísticas.

D. Validation Engine
Verifica:

- Campos obligatorios.
- Identificadores.
- Fechas.
- Valores.
- Coherencia entre campos.
- Duplicados.
- Reglas empresariales.

E. Odoo Connector
Se comunica con Odoo. Debe separar la lógica de la suite de las particularidades de Odoo. La suite no debe depender directamente de las tablas internas de Odoo.

F. Human Approval
Permite revisar operaciones antes de confirmarlas. Es especialmente importante en procesos contables y administrativos.

7.4. Estados documentales

Se propone manejar estados como:

```
RECIBIDO
   |
   v
PROCESANDO
   |
   v
EXTRAÍDO
   |
   v
VALIDADO
   |
   v
PENDIENTE DE APROBACIÓN
   |
   +-------> RECHAZADO
   |
   v
APROBADO
   |
   v
ENVIADO A ODOO
   |
   v
REGISTRADO
```

Los estados definitivos deberán ajustarse a las funcionalidades reales de RADIAN y al comportamiento de Odoo.

8. MÓDULO 5: INTEGRATION HUB

Este módulo será uno de los elementos más importantes de toda la arquitectura. Su responsabilidad será centralizar las conexiones entre Suite Empresarial Telcobras y los sistemas externos.

No debemos permitir que cada módulo implemente integraciones de manera desordenada y repetida. Por ejemplo, CRM, LeadGen y los futuros agentes podrían necesitar conectarse con WhatsApp. No sería conveniente que cada uno implemente por separado toda la comunicación con ese servicio.

El Integration Hub proporcionará conectores reutilizables y contratos comunes.

8.1. Integraciones contempladas

Sistema	Uso principal
Odoo	ERP, gestión administrativa y contable.
DIAN/RADIAN	Documentación electrónica.
WhatsApp	Comunicación empresarial.
Google Drive	Almacenamiento y consulta documental.
OneDrive	Almacenamiento y consulta documental.

Sistema	Uso principal
Shopify	Información comercial y pedidos.
Correo electrónico	Contacto y comunicación.
PostgreSQL	Integraciones con bases de datos autorizadas.
APIs externas	Conexión con otros servicios empresariales.

La disponibilidad concreta de cada conector dependerá de sus APIs, permisos y condiciones comerciales.

8.2. Componentes internos

Connector Registry
Catálogo de conectores disponibles.

Credential Manager
Administración segura de credenciales. Las credenciales no deben almacenarse en texto plano.

Integration Configuration
Permite asociar una integración con una organización. Ejemplo:

```
Empresa A
   |
   +---- Odoo
   +---- WhatsApp
   +---- Google Drive

Empresa B
   |
   +---- Odoo
   +---- Shopify
   +---- OneDrive
```

Cada empresa tendrá sus propias credenciales y configuraciones.

Integration Execution
Ejecuta las operaciones de comunicación con los sistemas externos.

Integration Logs
Registra:

- Solicitud.
- Destino.
- Fecha.
- Resultado.
- Tiempo de respuesta.
- Errores.
- Identificador de ejecución.

8.3. Regla arquitectónica

Los módulos no deben conectarse directamente a cualquier sistema externo de forma improvisada. Cuando una integración sea compartida por varios módulos, debe gestionarse mediante conectores reutilizables. Esto facilita mantenimiento, seguridad y sustitución de proveedores.

9. MÓDULO 6: API GATEWAY Y COMUNICACIÓN INTERNA

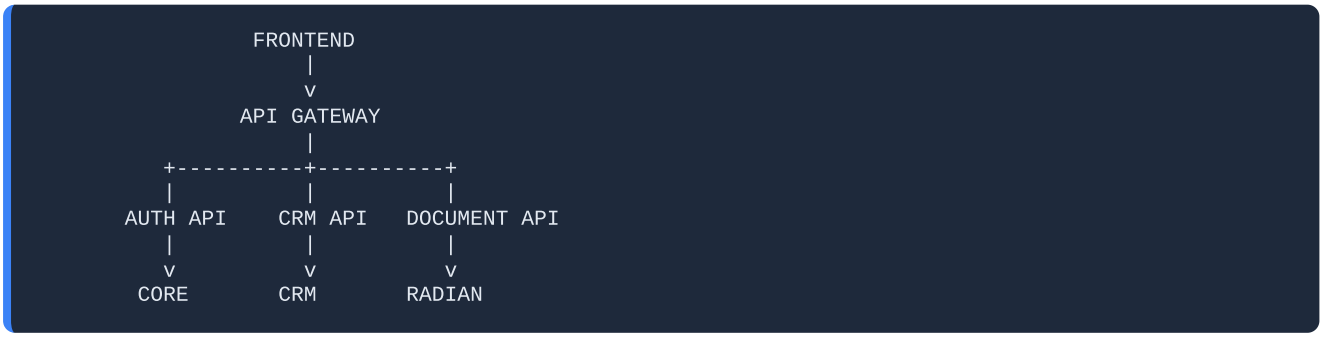
La suite necesitará una capa que permita controlar las solicitudes entre frontend, backend y servicios.

9.1. API Gateway

Será la puerta de entrada de las solicitudes externas hacia la plataforma. Responsabilidades:

- Enrutamiento.
- Validación de autenticación.
- Control de acceso.
- Rate limiting.
- Versionado de APIs.
- Registro de solicitudes.
- Control de errores.
- Correlación de peticiones.

Ejemplo:



9.2. Comunicación síncrona

Se utilizará cuando el usuario necesite una respuesta inmediata. Ejemplos:

- Consultar un cliente.
- Mostrar una factura.
- Consultar el estado de una integración.
- Obtener la configuración de un agente.

Tecnología propuesta: APIs REST sobre HTTPS.

9.3. Comunicación asíncrona

Se utilizará cuando una operación pueda tardar o deba ejecutarse en segundo plano. Ejemplos:

- Investigar cientos de empresas.
- Procesar documentos.
- Ejecutar campañas.
- Procesar tareas extensas.
- Ejecutar operaciones de agentes.
- Generar reportes.

Para esto se recomienda introducir una cola de trabajos. Redis puede ser una alternativa para determinados escenarios; también se podrán evaluar otros sistemas según los requisitos de persistencia, volumen y garantías de entrega. La elección definitiva no debe hacerse sin conocer las cargas de trabajo.

9.4. Eventos internos

Los módulos podrán publicar eventos de negocio. Ejemplos:

- lead.created
- lead.qualified
- customer.created
- document.received
- document.validated
- document.approval_requested
- document.approved
- integration.failed
- agent.execution.started
- agent.execution.completed

Estos nombres son ilustrativos. Los eventos permitirán que un módulo informe que algo ocurrió sin tener que conocer todos los módulos que necesitan reaccionar.

Ejemplo: Cuando LeadGen registra un prospecto, el CRM puede recibir el evento y procesarlo. Esto reduce el acoplamiento.

10. MÓDULO 7: AGENT MANAGEMENT

> Este módulo es fundamental para la visión futura, pero existe una decisión de orden que debe respetarse: La implementación definitiva de Agent Management y los agentes propios será la última gran etapa, después de que el resto de la suite esté funcionando. No debe adelantarse su construcción completa. La arquitectura conceptual sí debe contemplarlo desde el comienzo para evitar que posteriormente haya que reconstruir la suite.

10.1. Responsabilidad

Agent Management será el módulo encargado de administrar los agentes disponibles en la plataforma. No será necesariamente el lugar donde se ejecuta la inteligencia del agente. Su responsabilidad será administrar:

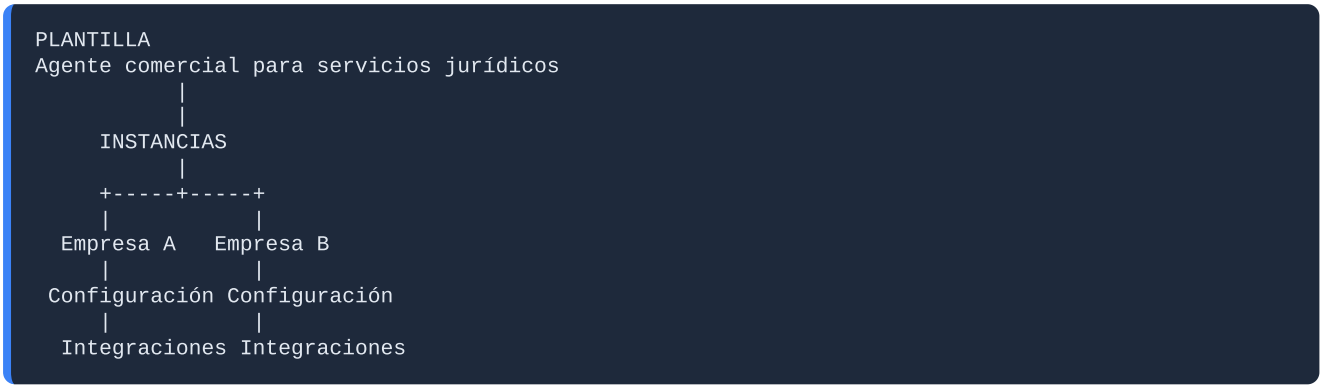
- Catálogo de agentes.
- Plantillas.
- Versiones.
- Instancias por empresa.
- Configuraciones.
- Permisos.
- Canales.
- Herramientas autorizadas.
- Conocimiento documental.
- Estado de disponibilidad.
- Historial de ejecuciones.
- Consumo.
- Motor de ejecución asignado.

10.2. Concepto de plantilla e instancia

Es importante separar tres elementos.

- **Plantilla de agente:** definición desarrollada y controlada por Telcobras.
- **Instancia de agente:** agente habilitado para una empresa.
- **Configuración:** parámetros específicos que la empresa puede modificar dentro de los límites permitidos.

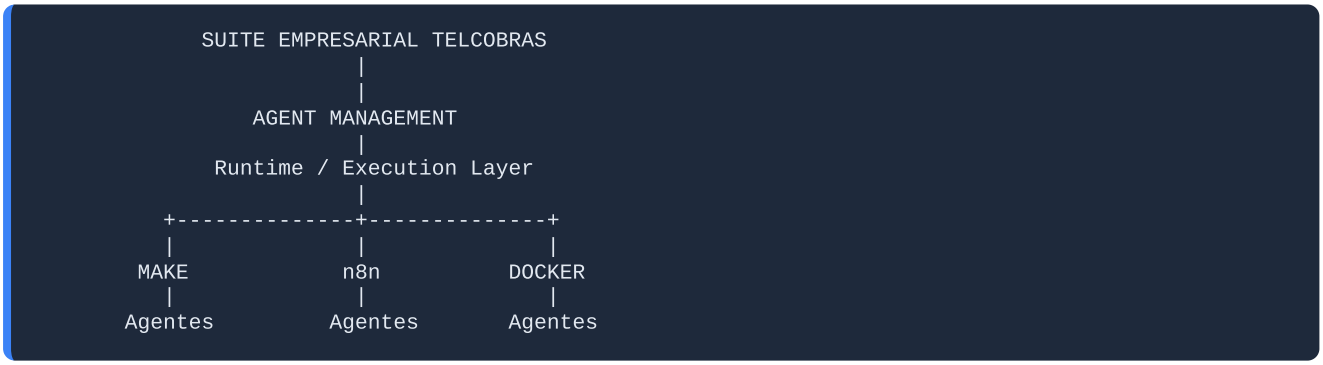
Ejemplo:



Dos empresas pueden utilizar la misma plantilla, pero tener configuraciones diferentes. No se deben mezclar sus conversaciones, credenciales, documentos ni historiales.

10.3. Runtime / Execution Layer

La capa Runtime será responsable de dirigir las solicitudes al motor que corresponda. Esta es la arquitectura acordada:



Hay una precisión importante para interpretar este esquema: Los tres motores representan las alternativas y etapas tecnológicas contempladas. No significa que todos deban permanecer como motores definitivos de producción.

Alternativa 1: Make.com
Make se utilizará como conexión temporal para los agentes que ya existen. La suite podrá comunicarse con escenarios mediante:

- Webhooks.
- Solicitudes HTTPS.
- Endpoints configurados.
- Respuestas estructuradas.

Make permitirá validar la integración entre la aplicación y los agentes existentes. La aplicación no debe exponer al usuario final la complejidad de los escenarios.

Alternativa 2: n8n
n8n se utilizará como entorno de construcción y validación de agentes. Aquí se podrán desarrollar y comprobar:

- Lógica.
- Integraciones.
- Herramientas.
- Flujos.
- Condiciones.
- Comportamientos.
- Resultados.

La decisión establecida es que n8n no será el motor definitivo de producción de los agentes propios. Se utilizará como entorno intermedio para validar la arquitectura y el comportamiento antes de reconstruir los agentes como software propio.

Importante sobre la licencia: la edición Community self-hosted de n8n tiene condiciones de uso específicas. Su documentación indica que el uso interno para construir y ejecutar workflows para clientes puede estar permitido bajo determinadas condiciones, mientras que permitir que clientes externos configuren workflows puede requerir licencia comercial. Aunque n8n no será el runtime definitivo, debe verificarse que el uso previsto durante el desarrollo y validación esté cubierto por la licencia. n8n Help Center

Alternativa 3: Docker
Docker será el entorno de ejecución de los agentes reconstruidos como aplicaciones propias. Los agentes ya no dependerán de Make ni de n8n para ejecutar su lógica principal. Se conectarán directamente con los modelos de IA y con las herramientas empresariales mediante APIs y servicios internos.

11. TECNOLOGÍA DEFINIDA PARA LOS AGENTES: OPENAI AGENTS SDK

La decisión tecnológica establecida es utilizar OpenAI Agents SDK para construir los agentes propios. Esta decisión es coherente con la intención de desarrollar agentes controlados por Telcobras y ejecutados dentro de su propia infraestructura.

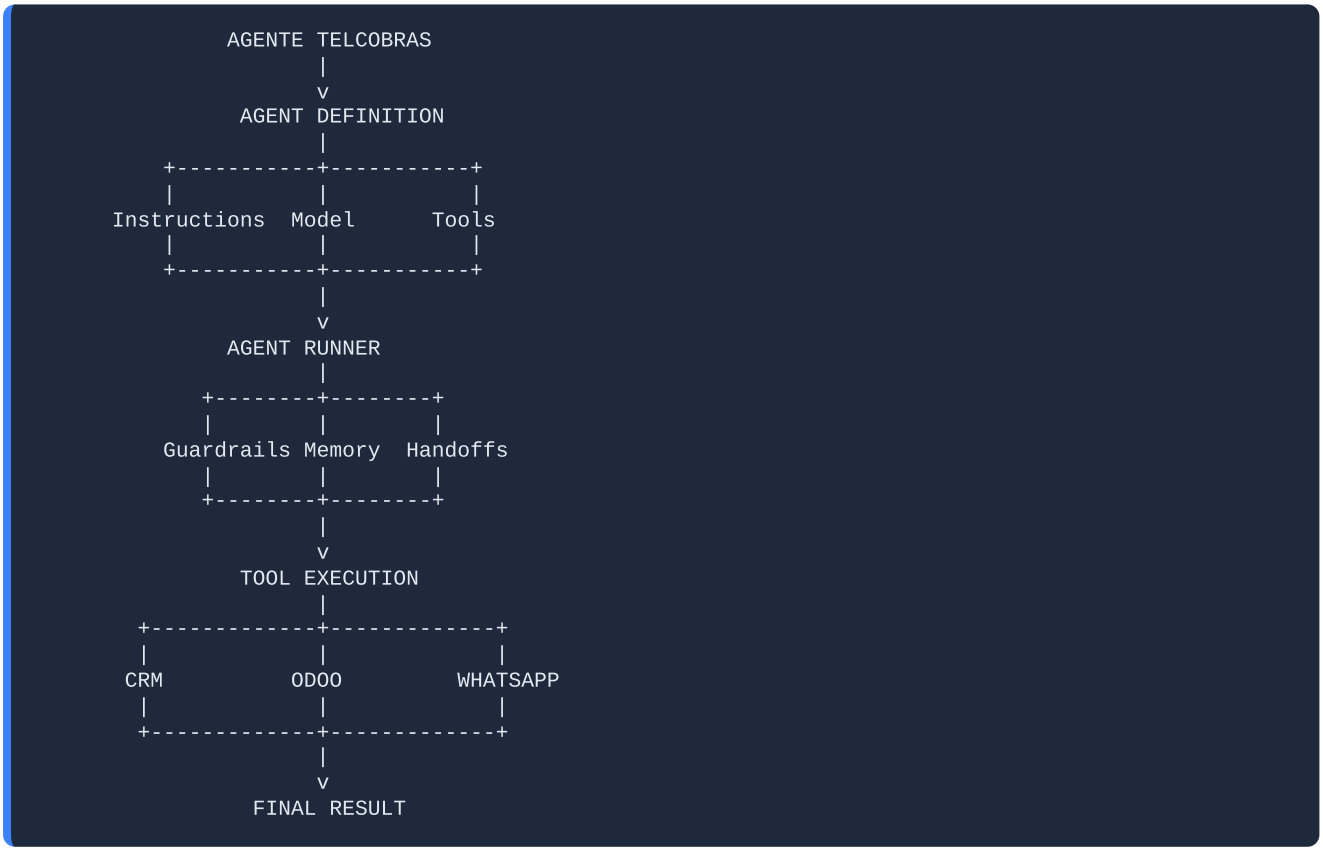
11.1. ¿Por qué utilizar Agents SDK?

El SDK ofrece componentes para construir agentes y coordinar procesos de varios pasos. Entre sus capacidades se encuentran:

- Definición de agentes.
- Instrucciones.
- Herramientas.
- Function calling.
- Transferencias entre agentes.
- Guardrails.
- Sesiones.
- Intervención humana.
- Trazabilidad.
- Orquestación de múltiples agentes.

El SDK ejecuta el ciclo del agente dentro de la aplicación desarrollada por Telcobras. No requiere que Make o n8n sean responsables de ejecutar la lógica central. OpenAI Developers

11.2. Arquitectura interna de un agente propio



11.3. Modelo de proveedores de IA

Aunque el SDK sea la tecnología principal de construcción, no debemos diseñar el sistema de forma que toda la suite quede acoplada permanentemente a un único modelo. La documentación del SDK contempla proveedores y configuraciones para trabajar con diferentes modelos y servicios compatibles. OpenAI Developers

Se propone crear una capa de configuración de modelos. Esta capa permitirá administrar:

- Proveedor.
- Modelo.
- Parámetros.
- Límites de uso.
- Credenciales.
- Disponibilidad.
- Política de selección.

La compatibilidad exacta de cada capacidad deberá verificarse con la versión del SDK y el proveedor elegido.

11.4. Tipos de agentes

La biblioteca podrá organizar agentes por industria y función. Ejemplos:

Industria	Tipo de agente
Servicios jurídicos	Atención y calificación de prospectos.
Comercio electrónico	Atención y consulta de pedidos.
Servicios profesionales	Captación y seguimiento comercial.
Contabilidad	Consulta documental y administrativa.
Educación	Atención de interesados y estudiantes.
Inmobiliario	Calificación y seguimiento de prospectos.

Son categorías conceptuales, no productos que deban desarrollarse simultáneamente.

11.5. Herramientas de los agentes

Los agentes no deberán tener acceso irrestricto a todos los sistemas. Cada agente dispondrá de herramientas autorizadas. Ejemplo:

```
AGENTE COMERCIAL
      |
      +---- consultar_cliente()
      |
      +---- buscar_productos()
      |
      +---- registrar_interaccion()
      |
      +---- crear_oportunidad()
      |
      +---- enviar_mensaje()
```

Las herramientas serán funciones controladas por backend o conectores autorizados. No se debe permitir que un agente construya libremente consultas SQL o ejecute operaciones administrativas sin validación.

11.6. Memoria de agentes

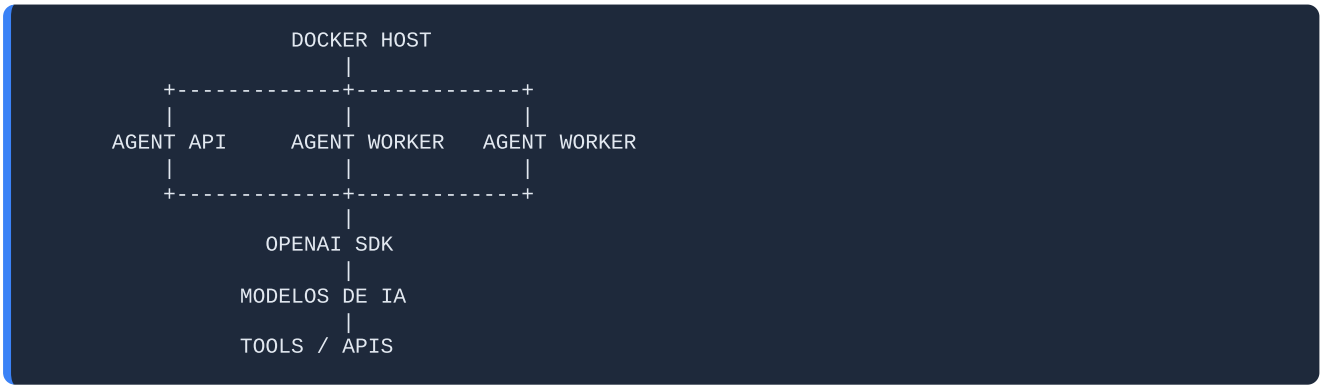
Se debe diferenciar:

- **Contexto conversacional:** información necesaria para mantener una conversación.
- **Memoria persistente:** información que debe mantenerse entre sesiones.
- **Datos empresariales:** información oficial almacenada en CRM, ERP o bases de datos.

La memoria de un agente no debe convertirse en la fuente oficial de información empresarial. Por ejemplo, el saldo de una factura debe consultarse en el sistema administrativo correspondiente, no confiarse a lo que el agente recuerda de una conversación anterior.

11.7. Ejecución en Docker

Los agentes podrán ejecutarse como servicios propios. Ejemplo conceptual:



No significa que cada agente deba tener obligatoriamente un contenedor independiente. Se podrá utilizar un runtime compartido que cargue distintas definiciones de agentes, siempre que existan aislamiento, límites y controles adecuados.

Docker Compose permite definir y administrar aplicaciones formadas por múltiples contenedores, redes y volúmenes, y contempla configuraciones específicas para producción. Docker Documentation

11.8. Observabilidad de los agentes

Cada ejecución debe permitir conocer:

- Qué agente se ejecutó.
- Qué organización lo solicitó.
- Qué versión utilizó.
- Qué modelo intervino.
- Qué herramientas fueron llamadas.
- Qué resultado produjo.
- Cuánto tardó.
- Qué errores ocurrieron.
- Cuál fue el consumo.

El Agents SDK incorpora tracing para registrar llamadas a modelos, herramientas, transferencias entre agentes y guardrails. Esta capacidad debe integrarse con la estrategia de observabilidad general de Telcobras. OpenAI GitHub

12. MÓDULO 8: DOCUMENT MANAGEMENT

Aunque RADIAN tendrá su propio procesamiento, conviene que la suite tenga una gestión documental transversal. No todos los documentos de las empresas provendrán de DIAN.

Podrán existir:

- Contratos.
- Propuestas.
- Certificados.
- Documentos comerciales.
- Archivos de clientes.
- Manuales.
- Documentos de conocimiento para agentes.
- Facturas.
- Soportes administrativos.

Por eso debe existir un servicio documental reutilizable.

Responsabilidades

- Almacenamiento.
- Clasificación.
- Metadatos.
- Asociación con entidades.
- Permisos.
- Versiones.
- Búsqueda.
- Descarga.
- Historial.
- Control de acceso.

El almacenamiento de archivos debe estar separado de la base de datos relacional. La base de datos almacenará metadatos y referencias; los archivos se conservarán en un servicio de almacenamiento de objetos o equivalente. Esto permitirá que RADIAN, CRM y Agent Management utilicen el mismo servicio documental sin duplicar archivos innecesariamente.

13. MÓDULO 9: NOTIFICACIONES Y COMUNICACIONES

Este módulo centralizará las notificaciones generadas por la suite. Canales posibles:

- Notificaciones internas.
- Correo electrónico.
- WhatsApp.
- Alertas administrativas.

Ejemplos:

- Nuevo lead registrado.
- Documento pendiente de aprobación.
- Error de integración.
- Agente desconectado.
- Nueva oportunidad comercial.
- Proceso documental finalizado.

La responsabilidad de este módulo será entregar notificaciones. No deberá convertirse en el propietario de toda la lógica comercial. Por ejemplo, CRM decide que existe una oportunidad nueva; Notifications se encarga de enviar la alerta correspondiente.

14. MÓDULO 10: ANALÍTICA Y REPORTES

La suite deberá permitir consultar información consolidada. Sin embargo, no todos los módulos deben realizar sus propios reportes de manera aislada. Se propone un dominio de analítica que consuma información autorizada de los módulos.

Indicadores posibles

Comerciales

- Leads generados.
- Prospectos calificados.
- Conversión.
- Ventas.
- Oportunidades abiertas.

Documentales

- Documentos procesados.
- Documentos pendientes.
- Errores de extracción.
- Aprobaciones pendientes.

Automatización

- Ejecuciones.
- Procesos completados.
- Procesos fallidos.
- Tiempo de ejecución.

Inteligencia artificial

- Ejecuciones de agentes.
- Consumo por empresa.
- Modelos utilizados.
- Herramientas invocadas.
- Tiempos de respuesta.
- Intervenciones humanas.

Los reportes deberán respetar los permisos y el aislamiento entre organizaciones.

15. MÓDULO 11: SUSCRIPCIONES, PLANES Y CONSUMO

Aunque no es necesario definir ahora un modelo comercial definitivo, la arquitectura debe contemplar la posibilidad de habilitar funcionalidades según el tipo de servicio contratado. Cada organización podrá tener:

- Plan.
- Estado.
- Módulos habilitados.
- Límites de uso.
- Consumo.
- Configuraciones comerciales.

El sistema deberá poder medir consumos como:

- Número de usuarios.
- Cantidad de documentos procesados.
- Investigaciones realizadas.
- Ejecuciones de agentes.
- Mensajes enviados.
- Almacenamiento utilizado.

No debe confundirse este módulo con la facturación electrónica empresarial. Una cosa es administrar el consumo de la plataforma Telcobras y otra es administrar la facturación de los clientes de las empresas usuarias.

16. MÓDULO 12: FUTURO ERP Y GESTIÓN ADMINISTRATIVA

La arquitectura podrá incorporar posteriormente funcionalidades adicionales. Entre ellas:

- Facturación.
- Cartera.
- Compras.
- Inventario.
- Proveedores.
- Gastos.
- Conciliación.
- Reportes financieros.

No se debe asumir que Telcobras desarrollará inmediatamente un ERP completo. En principio, la plataforma podrá integrarse con Odoo y otros sistemas externos mediante el Integration Hub. Si posteriormente se decide desarrollar funcionalidades ERP propias, deberán constituir un dominio independiente. La suite no debe duplicar funcionalidades de Odoo sin una decisión de producto y arquitectura que lo justifique.

17. ARQUITECTURA TÉCNICA GENERAL PROPUESTA

Esta sección traduce los módulos funcionales a componentes tecnológicos. Hay que distinguir las decisiones expresamente acordadas de las tecnologías recomendadas que todavía deben validarse.

17.1. Frontend

Propuesta tecnológica:

- React.
- TypeScript.
- Vite.
- Bootstrap o un sistema de componentes consistente.

Responsabilidades:

- Interfaz de usuario.
- Dashboard.
- Navegación.
- Gestión de módulos.
- Formularios.
- Tablas.
- Gráficos.
- Configuración empresarial.
- Administración de usuarios.
- Interfaz CRM.
- Interfaz documental.
- Futuro panel de agentes.

El frontend no debe contener lógica de negocio sensible ni conectarse directamente a las bases de datos. Todas las operaciones deben pasar por APIs autorizadas.

17.2. Backend central

Propuesta tecnológica inicial:

- Node.js.
- TypeScript.
- NestJS.

El backend será responsable de:

- Autenticación.
- Autorización.
- Organizaciones.
- Usuarios.
- CRM.
- Configuraciones.
- APIs.
- Integraciones.
- Auditoría.
- Coordinación de procesos.

NestJS es una propuesta, no una decisión irrevocable. Claude Code deberá investigar si resulta adecuado para la estructura real del proyecto antes de crear una implementación definitiva.

17.3. Servicios especializados en Python

Python puede mantenerse para servicios donde ya existe una base funcional o donde resulte técnicamente conveniente. Especialmente:

- LeadGen.
- Procesamiento documental existente.
- Futuros agentes con OpenAI Agents SDK, si se elige su implementación Python.

No se debe introducir Python y Node.js indiscriminadamente en todos los módulos. Cada lenguaje adicional genera costos de mantenimiento, dependencias, pruebas y despliegue.

17.4. Base de datos

Propuesta: PostgreSQL.

Responsabilidades:

- Usuarios.
- Organizaciones.
- Roles.
- Permisos.
- CRM.
- Leads.
- Oportunidades.
- Configuraciones.
- Auditoría.
- Metadatos documentales.
- Instancias de agentes.
- Ejecuciones.
- Integraciones.

Debe definirse una estrategia clara de aislamiento multiempresa.

17.5. Redis y procesamiento asíncrono

Se podrá utilizar Redis para:

- Caché.
- Colas de trabajos cuando la solución elegida lo soporte.
- Coordinación temporal.
- Control de tareas.
- Datos efímeros.

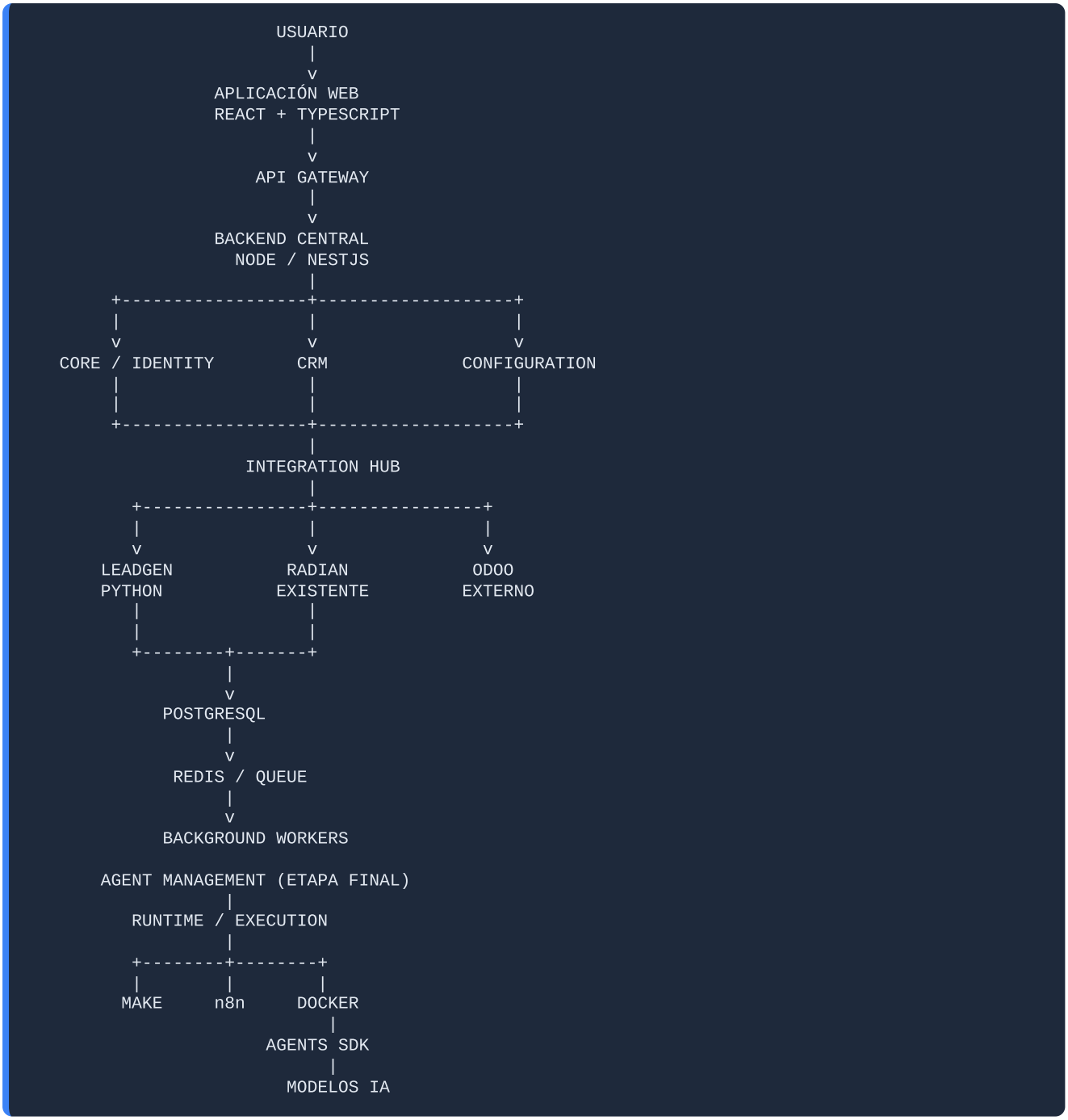
No debe utilizarse como sustituto indiscriminado de PostgreSQL.

17.6. Docker

Docker será la tecnología de contenedorización. Permitirá empaquetar servicios y mantener entornos reproducibles. Se utilizará para los servicios propios de la suite y, en la etapa correspondiente, para los agentes desarrollados con Agents SDK.

18. DIAGRAMA DE ARQUITECTURA TÉCNICA GENERAL

Este es el mapa técnico que conecta las capas.



Este esquema representa la arquitectura conceptual. No implica que todos los componentes deban estar implementados como microservicios independientes desde el primer momento.

19. MAPA DE DEPENDENCIAS ENTRE MÓDULOS

Esta sección es especialmente importante para Claude Code. No basta con conocer los módulos: hay que entender qué necesita cada uno.

Módulo	Depende de	Se relaciona con
Core	Base de datos	Todos los módulos.
Identity	Core	Todos los módulos protegidos.
Organization Management	Core	CRM, LeadGen, RADIAN, agentes.
RBAC	Identity	Todas las operaciones.
CRM	Core, Identity	LeadGen, notificaciones, analítica, agentes.
LeadGen	Core, Integration Hub	CRM, campañas, analítica.
RADIAN	Core, Document Management	DIAN, Odoo, aprobaciones, analítica.
Integration Hub	Core, Credential Manager	CRM, LeadGen, RADIAN, agentes.
Document Management	Core, Storage	RADIAN, CRM, futuros agentes.
Notifications	Core	CRM, RADIAN, integraciones, agentes.
Analytics	Core, fuentes autorizadas	Todos los dominios.
Agent Management	Core, Identity	CRM, Integration Hub, documentos.
Runtime Layer	Agent Management	Make, n8n y Docker.
Agentes propios	Runtime, Tools, Models	CRM, Odoo, documentos y canales.
Suscripciones	Core	Todos los módulos que consuman recursos.

Regla de dependencia

Los módulos especializados pueden utilizar servicios comunes, pero no deben crear dependencias circulares.

Ejemplo incorrecto:

```
CRM ---> LeadGen
 ^         |
 |         v
 +---------+
```

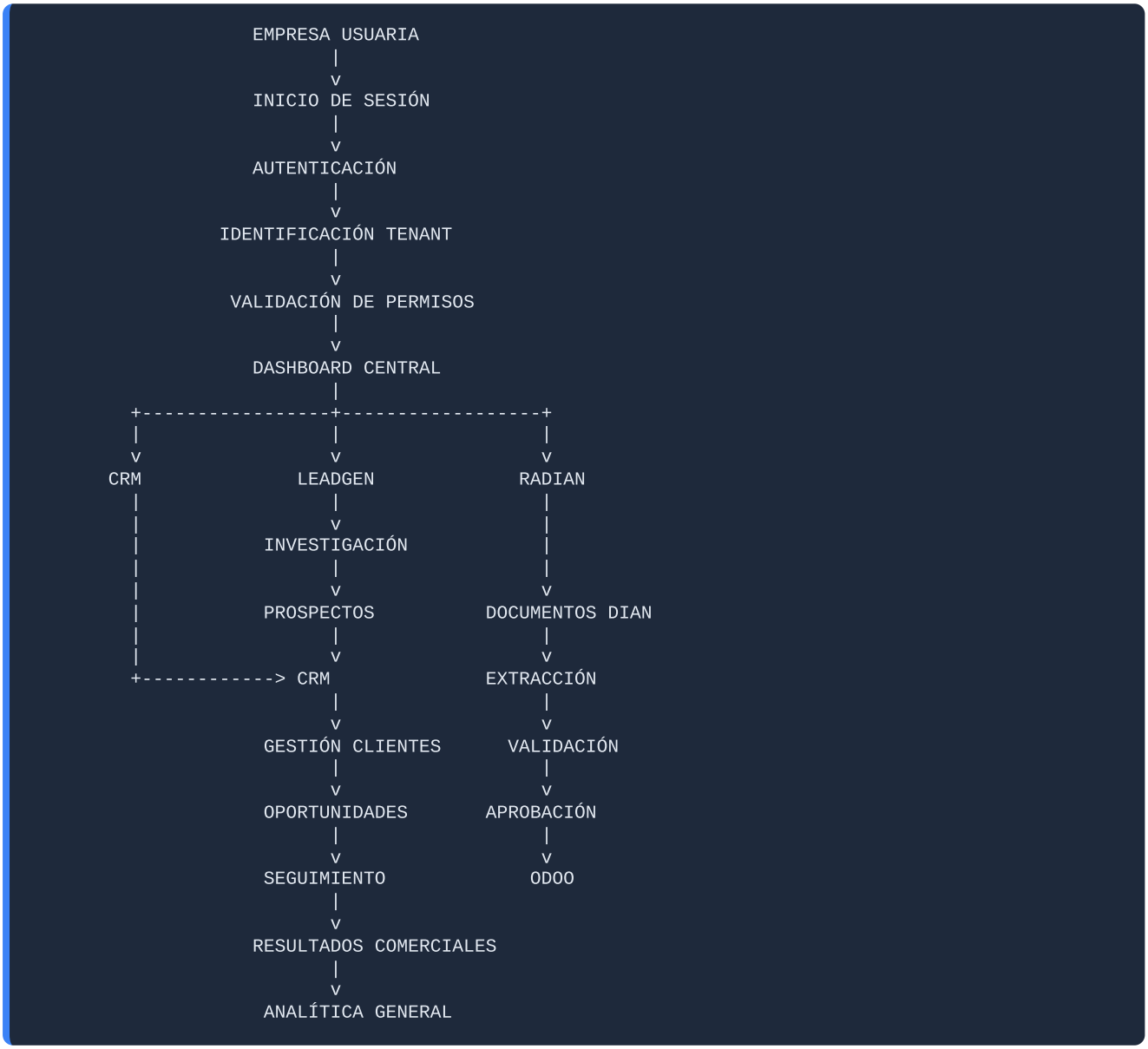
Ejemplo preferible:

```
LEADGEN
    |
    v
API / EVENTO
    |
    v
   CRM
```

LeadGen no necesita conocer la estructura interna del CRM. Solo necesita entregar información mediante un contrato definido.

20. DIAGRAMA DE FLUJO GENERAL DEL ECOSISTEMA

Este diagrama representa cómo podría funcionar la suite desde el punto de vista empresarial.



Los agentes se integrarán posteriormente como una capa de automatización inteligente conectada a los módulos autorizados.

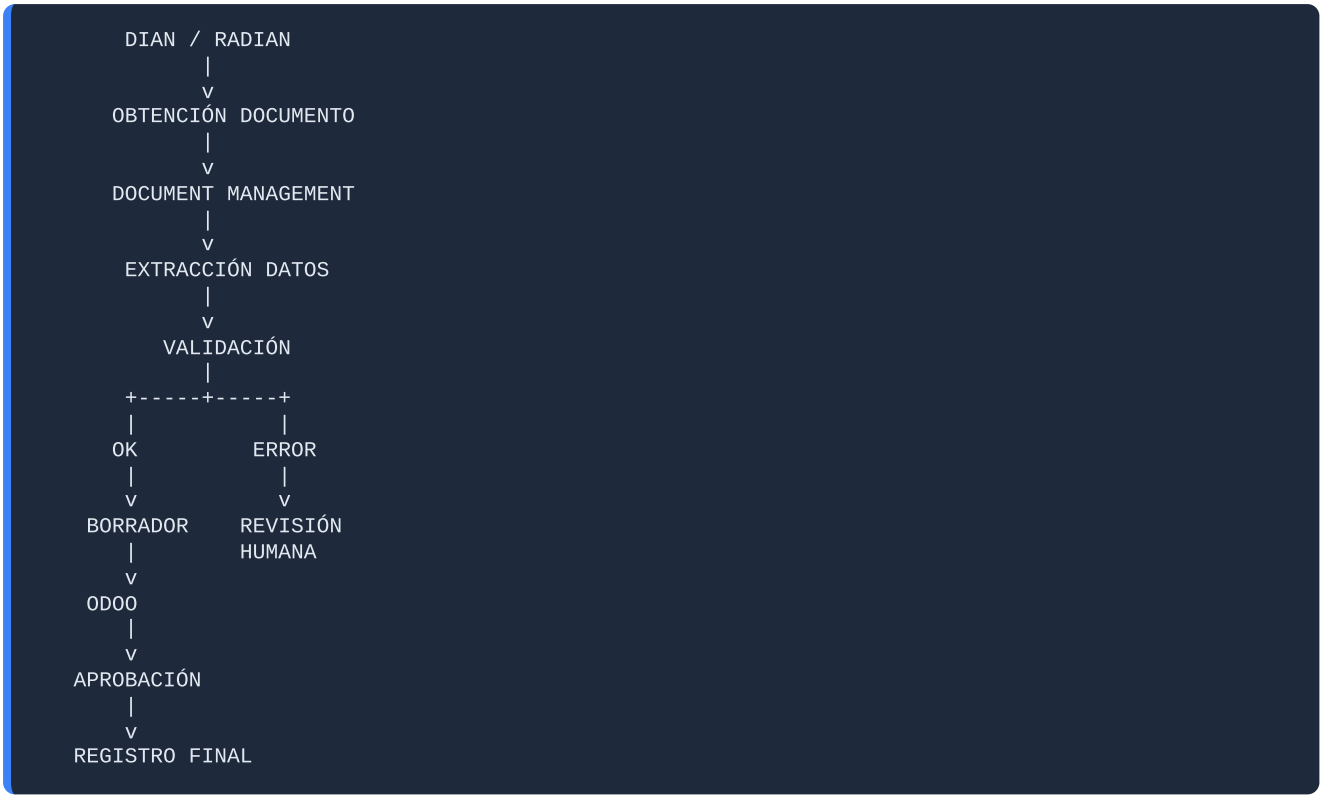
21. FLUJOS DE NEGOCIO ENTRE LOS MÓDULOS

Esta sección explica cómo se conectan las funcionalidades.

21.1. Flujo de captación comercial

```
SOLICITUD DE INVESTIGACIÓN
          |
          v
       LEADGEN
          |
          v
  BÚSQUEDA DE EMPRESAS
          |
          v
  EXTRACCIÓN DE INFORMACIÓN
          |
          v
    ENRIQUECIMIENTO
          |
          v
  NORMALIZACIÓN Y VALIDACIÓN
          |
          v
    DETECCIÓN DE DUPLICADOS
          |
          v
     RESULTADOS DE LEADGEN
          |
          v
          CRM
          |
          v
     REGISTRO DE LEADS
          |
          v
    CALIFICACIÓN COMERCIAL
          |
          v
      OPORTUNIDADES
          |
          v
     SEGUIMIENTO COMERCIAL
```

21.2. Flujo documental



21.3. Flujo de comunicación entre módulos

Cuando se incorpore Agent Management, los agentes podrán consultar información del CRM y ejecutar acciones mediante herramientas autorizadas. Ejemplo conceptual:



El agente no deberá tener acceso ilimitado a la base de datos ni a todas las funciones del sistema. Sus herramientas tendrán permisos definidos y se ejecutarán bajo el contexto de la organización correspondiente.

22. MODELO DE DATOS CONCEPTUAL

No se pretende definir todavía todas las tablas definitivas. Sin embargo, Claude Code necesita entender qué entidades principales existirán.

22.1. Entidades centrales



22.2. Entidades CRM

```
ORGANIZATION
    |
    +---- COMPANIES
    |        |
    |        +---- CONTACTS
    |
    +---- LEADS
    |        |
    |        +---- ACTIVITIES
    |
    +---- OPPORTUNITIES
             |
             +---- PIPELINE_STAGES
             |
             +---- ACTIVITIES
```

22.3. Entidades documentales

```
ORGANIZATION
    |
    +---- DOCUMENTS
    |        |
    |        +---- DOCUMENT_VERSIONS
    |        |
    |        +---- DOCUMENT_METADATA
    |
    +---- EXTRACTION_RESULTS
    |
    +---- VALIDATION_RESULTS
    |
    +---- APPROVAL_REQUESTS
```

22.4. Entidades de integraciones

```
ORGANIZATION
    |
    +---- CONNECTORS
    |
    +---- INTEGRATION_CONFIGURATIONS
    |
    +---- CREDENTIAL_REFERENCES
    |
    +---- INTEGRATION_EXECUTIONS
```

22.5. Futuras entidades de agentes

```
AGENT_TEMPLATE
      |
      +---- AGENT_VERSIONS
      |
      +---- AGENT_INSTANCES
                  |
                  +---- AGENT_CONFIGURATIONS
                  |
                  +---- AGENT_EXECUTIONS
                  |
                  +---- AGENT_SESSIONS
                  |
                  +---- AGENT_TOOLS
```

Una regla fundamental: todas las entidades empresariales que pertenezcan a una organización deberán tener un vínculo inequívoco con ella. La estrategia exacta de claves, restricciones, índices y relaciones se definirá al diseñar el esquema físico.

23. SEGURIDAD Y PROTECCIÓN DE LA INFORMACIÓN

La seguridad no puede agregarse al final. Debe formar parte de la arquitectura desde su concepción.

23.1. Aislamiento entre empresas

Una empresa nunca deberá consultar datos de otra. El aislamiento debe existir en:

- Backend.
- Base de datos.
- APIs.
- Archivos.
- Integraciones.
- Memoria de agentes.
- Historiales.
- Reportes.

23.2. Autorización

No basta con validar que un usuario haya iniciado sesión. También debe verificarse:

- A qué organización pertenece.
- Qué rol tiene.
- Qué recurso intenta consultar.
- Qué operación intenta ejecutar.
- Si tiene autorización sobre ese recurso.

OWASP identifica los problemas de autorización a nivel de objeto y autenticación entre los principales riesgos de seguridad de las APIs. Estos riesgos deben considerarse en el diseño del backend. OWASP Foundation

23.3. Protección de credenciales

Las claves API y credenciales externas no deben almacenarse:

- En el frontend.
- En archivos públicos.
- En repositorios Git.
- En registros de ejecución.
- En texto plano dentro de tablas.

Se deberá utilizar un mecanismo de secretos y cifrado apropiado.

23.4. Seguridad de agentes

Los agentes pueden realizar operaciones empresariales, por lo que necesitan controles específicos. Ejemplos:

- Consultar información.
- Crear clientes.
- Registrar oportunidades.
- Enviar mensajes.
- Crear borradores.
- Solicitar aprobaciones.

Las operaciones de mayor impacto deben contar con validaciones y aprobación humana cuando corresponda.

23.5. Protección contra instrucciones maliciosas

Los documentos y mensajes externos pueden contener instrucciones que intenten manipular el comportamiento de un agente. Por eso:

- El contenido documental debe tratarse como información, no como instrucciones privilegiadas.
- Los agentes solo tendrán herramientas necesarias.
- Las herramientas validarán los parámetros.
- Las operaciones sensibles tendrán controles externos al modelo.
- Los permisos no dependerán de lo que el modelo decida hacer.

24. ARQUITECTURA DE DESPLIEGUE E INFRAESTRUCTURA

La suite debe poder ejecutarse en servidores propios o alquilados. La infraestructura deberá permitir separar los servicios y escalar los componentes que lo necesiten.

24.1. Diagrama conceptual



Los agentes propios se incorporarán como servicios adicionales cuando llegue su etapa.

24.2. Ambientes

Se deben diferenciar:

- Desarrollo local.
- Pruebas.
- Staging.
- Producción.

Cada ambiente deberá tener configuraciones y credenciales independientes. No se deben utilizar datos reales de clientes en pruebas sin las autorizaciones y controles correspondientes.

24.3. Contenedores

Los servicios que se empaqueten en Docker deberán tener:

- Imagen reproducible.
- Dependencias controladas.
- Variables de entorno.
- Health checks.
- Logs.
- Política de reinicio.
- Configuración de red.
- Límites de recursos cuando corresponda.

Docker Compose puede ser suficiente para administrar varios servicios en una primera infraestructura controlada. La necesidad de una plataforma de orquestación más compleja dependerá del crecimiento y de los requisitos operativos. Docker Documentation

25. PRINCIPIOS DE INGENIERÍA QUE DEBEN RESPETARSE

Estas reglas son transversales a todo el desarrollo.

1. **Modularidad**
   Cada módulo debe tener responsabilidades claras.
2. **Bajo acoplamiento**
   Los módulos no deben depender de detalles internos de otros módulos.
3. **Contratos de API**
   Toda comunicación debe tener estructuras y respuestas definidas.
4. **Multi-tenancy desde el inicio**
   No se debe construir una aplicación para una sola empresa y tratar de convertirla en multiempresa posteriormente.
5. **Seguridad por defecto**
   Toda operación debe requerir autenticación y autorización según corresponda.
6. **Trazabilidad**
   Las operaciones empresariales importantes deben poder auditarse.
7. **Idempotencia**
   Cuando una operación pueda recibirse varias veces, el sistema debe evitar ejecutar accidentalmente la misma acción más de una vez. Por ejemplo, un webhook recibido dos veces no debe crear dos facturas idénticas.
8. **Manejo de errores**
   Los servicios deben devolver errores estructurados y registrar información suficiente para diagnosticar problemas.
9. **Procesamiento asíncrono**
   Los procesos extensos no deben bloquear innecesariamente las solicitudes web.

10. **Separación entre datos y archivos**
    Los archivos deben almacenarse mediante un sistema documental; la base de datos mantendrá sus metadatos.
11. **Configuración externa**
    Las configuraciones por ambiente no deben quedar incrustadas en el código.
12. **Versionamiento**
    Las APIs y los componentes que puedan cambiar deben contar con mecanismos de versionamiento.
13. **Pruebas**
    Cada módulo debe contar con pruebas unitarias, de integración y de sus flujos críticos.
14. **Documentación**
    Los contratos, decisiones y componentes deben documentarse para facilitar mantenimiento y evolución.

26. DECISIONES DE ARQUITECTURA: QUÉ ESTÁ ACORDADO Y QUÉ DEBE INVESTIGARSE

Es importante que Claude Code no interprete todas las propuestas de este documento como decisiones irrevocables. Esta tabla separa las decisiones expresas de las propuestas técnicas.

Elemento	Estado	Decisión o criterio
Nombre del producto	Acordado	Suite Empresarial Telcobras.
Aplicación web central	Acordado	Será el centro de acceso y administración.
Arquitectura modular	Acordado	Módulos especializados conectados mediante contratos.
LeadGen	Acordado	Se integrará como módulo propio.
RADIAN	Acordado	Se integrará como módulo documental.
CRM	Acordado conceptualmente	Será el centro de gestión comercial.
Multiempresa	Requisito arquitectónico	Separación estricta de datos y permisos.
Agent Management	Acordado conceptualmente	Administrará los agentes.
Runtime / Execution Layer	Acordado conceptualmente	Desacoplará la suite de los motores de ejecución.
Make	Acordado	Integración temporal con agentes existentes.

Elemento	Estado	Decisión o criterio
n8n	Acordado	Entorno de construcción y validación.
Docker	Acordado	Entorno de ejecución de agentes propios.
OpenAI Agents SDK	Acordado	Tecnología para construir los agentes propios.
React + TypeScript	Propuesta técnica	Debe validarse con las necesidades reales del frontend.
Vite	Propuesta técnica	Herramienta de construcción del frontend.
NestJS	Propuesta técnica	Backend central sugerido.
Python	Tecnología existente/propuesta	Mantener servicios que ya lo utilizan; evaluar Agents SDK en Python.
PostgreSQL	Propuesta técnica fuerte	Base de datos relacional principal.
Redis	Propuesta técnica	Caché y procesamiento asíncrono según necesidad.
Docker Compose	Propuesta técnica	Gestión inicial de servicios contenedorizados.
Odoo	Integración existente	Mantenerlo como sistema externo integrado.
Arquitectura de microservicios	Pendiente	No convertir todos los módulos en servicios independientes sin justificación.
Proveedor de almacenamiento	Pendiente	Evaluar opciones de almacenamiento de objetos.
Proveedor de autenticación	Pendiente	Evaluar implementación propia o proveedor especializado.
Cola de trabajos	Pendiente	Seleccionar según necesidades de procesamiento.
Modelo comercial	Pendiente	Definir planes, límites y consumo.
Tecnología de analítica	Pendiente	Dependerá de los reportes y volumen de información.

27. DECISIONES IMPORTANTES SOBRE AGENTES QUE NO DEBEN CAMBIARSE

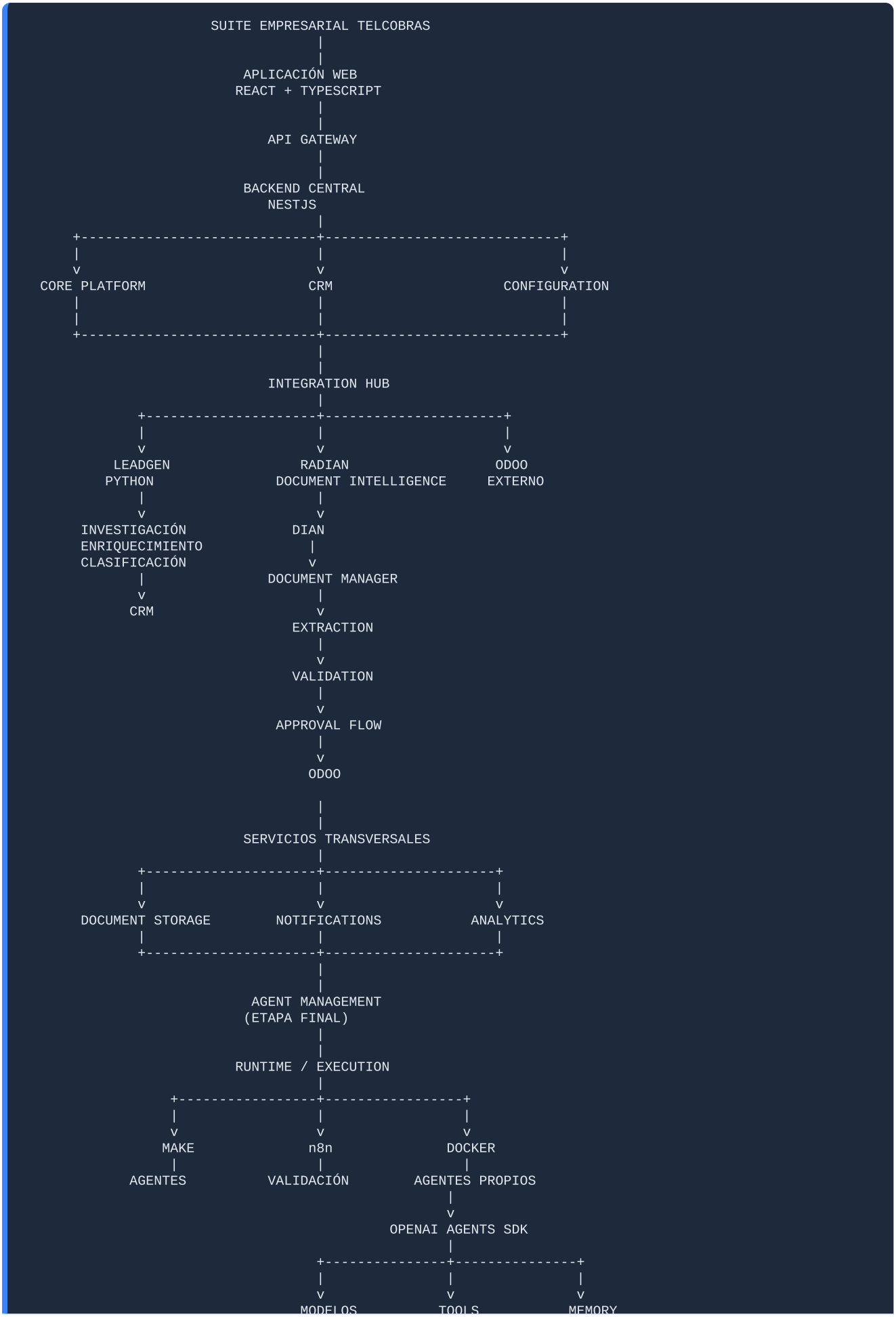
Esta sección merece especial atención porque resume lo que ya se ha definido.

1. La etapa de agentes propios será la última gran etapa del desarrollo de la suite.
2. Primero debe funcionar el resto de la plataforma.
3. Los agentes existentes en Make podrán conectarse temporalmente mediante webhooks o HTTPS.
4. n8n se utilizará para construir y validar agentes.
5. n8n no será el motor definitivo de producción de los agentes propios.
6. Los agentes validados se reconstruirán como aplicaciones propias.
7. La tecnología principal para construir esos agentes será OpenAI Agents SDK.
8. Los agentes propios se ejecutarán en Docker.
9. La suite se comunicará con los agentes a través de Agent Management y Runtime / Execution Layer.
10. Los usuarios finales no tendrán acceso libre a la construcción de agentes.
11. Telcobras controlará las plantillas, versiones, capacidades y herramientas.
12. Las empresas podrán personalizar parámetros autorizados de sus agentes.

> **Importante:** el diagrama Make / n8n / Docker representa el ecosistema de motores contemplados, pero la estrategia acordada es que Docker sea el entorno definitivo de los agentes reconstruidos.

28. DIAGRAMA MAESTRO COMPLETO

Este es el mapa general que reúne todos los dominios de la arquitectura.


SUITE EMPRESARIAL TELCOBRAS
APLICACIÓN WEB
REACT + TYPESCRIPT
API GATEWAY
BACKEND CENTRAL
NESTJS
CORE PLATFORM
CRM
CONFIGURATION
INTEGRATION HUB
LEADGEN
PYTHON
INVESTIGACIÓN
ENRIQUECIMIENTO
CLASIFICACIÓN
CRM
RADIAN
DOCUMENT INTELLIGENCE
DIAN
DOCUMENT MANAGER
EXTRACTION
VALIDATION
APPROVAL FLOW
ODOO
ODOO
EXTERNO
SERVICIOS TRANSVERSALES
DOCUMENT STORAGE
NOTIFICATIONS
ANALYTICS
AGENT MANAGEMENT
(ETAPA FINAL)
RUNTIME / EXECUTION
MAKE
AGENTES
n8n
VALIDACIÓN
DOCKER
AGENTES PROPIOS
OPENAI AGENTS SDK
MODELOS
TOOLS
MEMORY

29. INSTRUCCIONES PARA CLAUDE CODE: CÓMO DEBE INTERPRETAR ESTE DOCUMENTO

Esta sección está dirigida directamente a Claude Code. El documento anterior define la visión arquitectónica y funcional de Suite Empresarial Telcobras. Claude Code debe actuar como asistente de ingeniería de software, investigación técnica y apoyo a la implementación. La dirección arquitectónica y las decisiones de producto pertenecen al responsable del proyecto. Claude Code no debe tomar decisiones estructurales importantes unilateralmente.

29.1. No asumir que el repositorio está vacío

Antes de proponer implementaciones, debe inspeccionar los repositorios y activos disponibles. En particular:

- LeadGen.
- RADIAN.
- Automatizaciones existentes.
- Integraciones con Odoo.
- Escenarios de Make.
- Código de frontend o backend que ya exista.

No debe reescribir componentes funcionales sin analizar primero su arquitectura.

29.2. No desarrollar toda la suite de una sola vez

Este documento describe el ecosistema completo. No significa que se deban implementar todos los módulos simultáneamente. La implementación debe responder a las decisiones del arquitecto del proyecto. No se deben inventar prioridades, cronogramas o fases diferentes a las que se hayan acordado.

29.3. No confundir propuesta con decisión definitiva

Cuando una tecnología aparezca como propuesta, Claude Code debe:

1. Investigar.
2. Comparar alternativas.
3. Evaluar compatibilidad.
4. Identificar ventajas y desventajas.
5. Presentar una recomendación.
6. Esperar aprobación cuando la decisión afecte la arquitectura.

29.4. Mantener documentación técnica

Se recomienda que el repositorio incluya documentación como:

```
/docs
   |
   +---- architecture
   |       |
   |       +---- overview.md
   |       +---- system-context.md
   |       +---- module-map.md
   |       +---- data-flow.md
   |
   +---- decisions
   |       |
   |       +---- ADR-001-architecture-style.md
   |       +---- ADR-002-database.md
   |       +---- ADR-003-authentication.md
   |
   +---- modules
   |       |
   |       +---- core.md
   |       +---- crm.md
   |       +---- leadgen.md
   |       +---- radian.md
   |       +---- integration-hub.md
   |       +---- agent-management.md
   |
   +---- api
   |
   +---- database
   |
   +---- security
   |
   +---- deployment
```

Los ADR (Architecture Decision Records) permitirán registrar las decisiones importantes, su justificación y sus consecuencias.

29.5. No mezclar módulos por comodidad

Claude Code debe evitar soluciones que concentren toda la lógica en un único archivo, controlador o servicio. La organización del código debe reflejar los dominios funcionales.

29.6. No generar complejidad innecesaria

La arquitectura debe ser escalable, pero no se debe implementar infraestructura distribuida compleja sin necesidad. Por ejemplo:

- No introducir Kubernetes solo porque el proyecto utilizará Docker.
- No crear decenas de microservicios sin una justificación.
- No duplicar APIs.
- No implementar dos sistemas de autenticación.
- No crear una base de datos independiente por módulo sin analizar sus ventajas y costos.

La arquitectura debe crecer con el producto.

29.7. Seguridad obligatoria

Claude Code debe verificar permanentemente:

- Aislamiento multiempresa.
- Autenticación.
- Autorización.
- Validación de entradas.
- Protección de credenciales.
- Manejo de errores.
- Auditoría.
- Control de acceso a documentos.
- Seguridad de integraciones.
- Seguridad de herramientas de agentes.

29.8. Investigar documentación oficial

Cuando se requiera implementar una tecnología, debe priorizar documentación oficial y versiones actuales. Especialmente:

- OpenAI Agents SDK.
- PostgreSQL.
- Docker.
- React.
- TypeScript.
- NestJS.
- Redis.
- Odoo.
- APIs de DIAN.
- Make.
- n8n.

No debe basar decisiones críticas exclusivamente en ejemplos antiguos, publicaciones de terceros o código generado sin verificar.

30. REFERENCIAS TÉCNICAS OFICIALES PARA LA INVESTIGACIÓN

Estas referencias sirven como punto de partida para profundizar en las tecnologías principales.

Inteligencia artificial

- OpenAI Agents SDK — documentación general
- OpenAI Agents SDK — Python
- OpenAI Agents SDK — modelos y proveedores
- OpenAI Agents SDK — tracing y observabilidad

Automatización

- n8n — condiciones de licencia y uso

Bases de datos

- PostgreSQL — Row-Level Security

Infraestructura

- Docker Compose
- Docker Compose en producción

Seguridad

- OWASP API Security Top 10
- Superintendencia de Industria y Comercio — Ley 1581 de 2012

31. CONSIDERACIONES FINALES DE ARQUITECTURA

Hay cinco ideas que deben permanecer en el centro del desarrollo.

1. **La suite es el producto, los módulos son sus capacidades**
   LeadGen, RADIAN, CRM y los futuros agentes no deben convertirse en aplicaciones desconectadas. Son capacidades de una misma plataforma empresarial.
2. **Los activos existentes deben aprovecharse**
   LeadGen y RADIAN ya tienen desarrollo funcional. La estrategia no debe consistir en desechar lo que existe, sino en integrarlo correctamente, respetando su lógica y mejorando su arquitectura cuando sea necesario.
3. **La interoperabilidad es fundamental**
   El valor de la plataforma no está solamente en que cada módulo funcione. Está en que los módulos puedan compartir información y producir resultados conjuntos. Por ejemplo: LeadGen encuentra una empresa → CRM registra el prospecto → una oportunidad comercial se genera → un agente puede realizar seguimiento → Odoo puede intervenir cuando exista una operación administrativa. Este es el tipo de integración que debe caracterizar a Telcobras.

4. **Los agentes deben ser una capacidad propia, no una dependencia permanente**
   La decisión de utilizar Make y n8n como herramientas intermedias y desarrollar posteriormente agentes propios mediante OpenAI Agents SDK y Docker es coherente con el objetivo de controlar la tecnología. La suite debe quedar desacoplada de los motores de ejecución.
5. **La arquitectura debe permitir crecer sin perder el control**
   El producto será desarrollado con ayuda de herramientas avanzadas de codificación mediante IA. Por tanto, la documentación, los contratos, las pruebas y los registros de decisiones tendrán una importancia especial. La IA podrá acelerar la implementación, pero las decisiones de arquitectura, seguridad y negocio deben seguir siendo verificadas y aprobadas por el responsable del proyecto.

CONCLUSIÓN GENERAL

Suite Empresarial Telcobras se plantea como una plataforma empresarial modular que reúne inteligencia comercial, CRM, gestión documental, integraciones empresariales y, posteriormente, agentes de inteligencia artificial propios.

Su núcleo central administrará organizaciones, usuarios, permisos y configuraciones.

Los módulos especializados realizarán las operaciones correspondientes:

- LeadGen: investigación y captación de prospectos.
- CRM: administración comercial.
- RADIAN: obtención y procesamiento documental.
- Integration Hub: comunicación con sistemas externos.
- Document Management: almacenamiento y gestión de archivos.
- Notifications: comunicaciones y alertas.
- Analytics: indicadores y reportes.
- Agent Management: administración futura de agentes.
- Runtime / Execution Layer: desacoplamiento de los motores de ejecución.

La infraestructura estará basada en servicios web, APIs, una base de datos relacional, procesamiento asíncrono y contenedores.

La arquitectura no busca construir un sistema enorme e innecesariamente complejo desde el primer día. Busca establecer una estructura sólida que permita incorporar funcionalidades progresivamente, aprovechar los desarrollos existentes y mantener el control tecnológico del producto.

La visión final es que Telcobras no sea simplemente una empresa que configura herramientas de terceros, sino una empresa que desarrolla y controla una plataforma tecnológica propia, capaz de integrar procesos empresariales y automatizarlos mediante software e inteligencia artificial.

Mi recomendación para utilizar este documento con Claude Code

No le entregaría este informe acompañado de una orden para que empiece a programar toda la suite. Lo utilizaría primero como documento de contexto permanente del proyecto.

Puedes guardar este informe en el repositorio con el nombre:

```
docs/ARQUITECTURA_GENERAL_TELCOBRAS.md
```

Y acompañarlo con esta instrucción inicial:

```
Este documento define la visión arquitectónica general de Suite Empresarial
Telcobras.

Quiero que actúes como arquitecto de software senior, investigador técnico
y asistente de ingeniería.

Tu responsabilidad es ayudarme a desarrollar esta plataforma respetando
la arquitectura, los módulos, las decisiones y las restricciones descritas
en el documento.

Yo soy el responsable de arquitectura y tomaré las decisiones finales.

INSTRUCCIONES:

1. No comiences a desarrollar toda la plataforma.
2. Primero comprende la arquitectura completa.
3. Identifica las decisiones confirmadas y las que todavía requieren
   investigación.
4. No inventes tecnologías ni detalles de implementaciones existentes.
5. Cuando necesites información sobre LeadGen o RADIAN, solicita e
   inspecciona sus repositorios.
6. Investiga la documentación oficial de cada tecnología antes de
   recomendar implementaciones.
7. Evita crear complejidad arquitectónica innecesaria.
8. Mantén la separación entre módulos.
9. Respeta el aislamiento multiempresa desde el diseño.
10. No adelantes la implementación de Agent Management y los agentes
    propios: esa etapa será la última.
11. No modifiques decisiones arquitectónicas importantes sin presentarme
    primero los motivos y las alternativas.
12. Mantén actualizada la documentación técnica y los ADR.

Por ahora, tu primera responsabilidad es comprender el sistema completo,
detectar inconsistencias, identificar decisiones pendientes y ayudarme
a investigar la arquitectura antes de implementar componentes.
```

Una última observación importante: este documento es una arquitectura de referencia inicial, no una especificación técnica cerrada. Las decisiones de stack, infraestructura, seguridad, contratos de API y modelo de datos deben convertirse progresivamente en decisiones técnicas verificadas. Esa distinción evitará que Claude Code convierta una propuesta conceptual en una implementación rígida que después resulte costosa de modificar.